

Exception-Handling – Teil 2

Polymorphie und Regeln

Programmierung 1

PIB-PR1 / DFIW-PRG1

Software Engineering und Software Quality Assurance { SESQA }

Prof. Dr. Martin Burger { 22. Januar 2026 }

noan



Plan der Präsenzveranstaltungen

VW	KW	 Inhalte in der „Vorlesung“	Mo	Di	Mi	Do
1	42	Willkommen und erste Orientierung, Lebenslanges Lernen				
2	43	Einstieg in Java (Kap. 1)				
3	44	Teamarbeit				
4	45	Klassen und Objekte (Kap. 2)				
5	46	Elementare Datentypen und Referenzen (Kap. 3)				
6	47	Methoden benutzen Instanzvariablen (Kap. 4)				
7	48	Ein Programm schreiben (Kap. 5)				
8	49	Die Java-API kennenlernen (Kap. 6)				
9	50	Vererbung und Polymorphie (Kap. 7)				
10	51	Interfaces und abstrakte Klassen (Kap. 8)				
	52	Vorlesungsfreie Zeit (Jahreswechsel)				
	01	Vorlesungsfreie Zeit (Jahreswechsel)				
11	02	Konstruktoren und Garbage Collection (Kap. 9)				
12	03	Zahlen und Statisches (Kap. 10)				
13	04	Exception-Handling (Kap. 13)				
14	05	Serialisierung und Datei-I/O (Kap. 16)				
15	06	Fragen und Antworten (Puffer)				

„Kein Plan 
überlebt den
ersten
Kontakt  mit
der Realität .

Frei nach Helmuth Karl
Bernhard von Moltke

Exception-Handling

 Plan für diese Woche

 Houston, wir haben eine Exception

 Geprüfte und Ungeprüfte

 Flusskontrolle

 finally-Block

 Pair Programming

 Mehrere Exceptions

 Polymorphe Exceptions

 Ausweichen

 Exception-Regeln

 Spaß

Exception-Handling

Verbindungen herstellen

 Tauschen Sie sich mit einer Nachbar:in aus!

 Wie lässt sich erreichen, dass eine Methode mehrere Arten von Ausnahmen werfen kann?

 Wie wirkt sich Polymorphie auf die Behandlung von Ausnahmen aus?

 Was bedeutet es, einer Ausnahme „auszuweichen“?

 Welche Kombinationen von try/catch/finally sind erlaubt?

 Welche Fragen haben Sie zu diesem Kapitel?

13 Exception-Handling

Riskantes Verhalten



Dinge passieren. Eine Datei fehlt. Der Server ist down. Egal wie gut Sie als Programmierer sind, Sie können nicht alles unter Kontrolle haben. Dinge können schiefgehen. *Richtig* schief. Wenn Sie eine riskante Methode schreiben, brauchen Sie Code, der mit den schlimmen Dingen umgeht, die passieren könnten. Aber woher wissen Sie, *wann* eine Methode riskant ist? Und wo tun Sie den Code hin, der diese **Ausnahmesituation** *behandelt*? Bisher sind wir in diesem Buch keine *echten* Risiken eingegangen. Klar, es sind zur Laufzeit ein paar Dinge danebengegangen, aber der Grund waren meistens Fehler in unserem eigenen Code. Programmierfehler, *Bugs* eben. Und die sollten wir während der Entwicklungsphase reparieren. Nein, hier geht es um Code, für den Sie *nicht* garantieren können, dass er zur Laufzeit funktioniert. Code, der erwartet, dass eine Datei im richtigen Verzeichnis liegt, dass der Server läuft oder dass der Thread nicht aufwacht. Und wir müssen uns *jetzt* darum kümmern. Denn in *diesem* Kapitel bauen wir etwas, das die riskante JavaSound-API verwendet. Wir programmieren ein Programm zum Abspielen von MIDI-Dateien.

Hier fängt ein neues Kapitel an 421



Mehrere Exceptions

```
1 import java.io.IOException;
2 import java.nio.file.Files;
3 import java.nio.file.Paths;
4 import java.text.NumberFormat;
5 import java.text.ParseException;
6 import java.util.Locale;
7
8 public class MultipleExceptDemo {
9
10     // Declare the *two* checked exceptions that might be thrown:
11     public static Number readAndParseUSD(String filePath) throws IOException, ParseException {
12         // Might throw IOException:
13         String content = Files.readString(Paths.get(filePath)).trim();
14
15         NumberFormat currencyFormat = NumberFormat.getCurrencyInstance(Locale.US);
16
17         // Might throw ParseException:
18         return currencyFormat.parse(content);
19     }
20
21     public static void main(String[] args) {
22         String filePath = "money.txt";
23
24         try {
25             Number amount = readAndParseUSD(filePath);
26             System.out.println("The amount is: " + amount);
27         } catch (IOException e) {
28             // java.lang.Exception <- java.io.IOException
29             System.err.println("Could not read from file due to: " + e.getClass().getName());
30             System.err.println("Message: " + e.getMessage());
31         } catch (ParseException e) {
32             // java.lang.Exception <- java.text.ParseException
33             System.err.println("Could not parse file contents due to: " + e.getClass().getName());
34             System.err.println("Message: " + e.getMessage());
35         }
36     }
37 }
38
```

STDIN

Input for the program (Optional)

Output:

Click on RUN button to see the output

! **Funktioniert
im OneCompiler
nicht (mehr).**

„Meine Methode kann auch mehrere Exceptions werfen. Dazu deklarriere ich dann alle überprüften (checked) Exceptions, die sie werfen könnte.“

Merle – aufmerksame Student:in im 1. Semester



„Der Compiler stellt sicher, dass ich alle überprüften Exceptions, die von deiner Methode ausgelöst werden, auch behandle. Die dafür nötigen catch-Blöcke schreibe ich unter meinem try-Block hintereinander.“

Gerit – aufgeweckte Student:in im 1. Semester



! ? Quiz

🧑 Mehrere Exceptions

- 🤔 Ich mache einige Aussagen – sind diese Aussagen richtig oder falsch?
- 👍 Bei **richtigen** Aussagen **stehen** Sie **kurz auf**!
- 👎 Bei **falschen** Aussagen **bleiben** Sie **sitzen**!



Ein einzelner `try`-Block kann viele verschiedene `catch`-Blöcke haben.

Richtig!

Eine Methode kann nur eine Art von
Exception werfen.

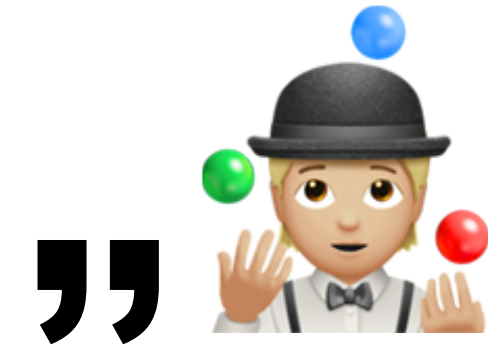
Eine Methode kann mehr als eine Exception werfen.

Wenn eine Methode mehr als eine Exception wirft, muss sie auch alle – zumindest überprüften (checked) – deklarieren.

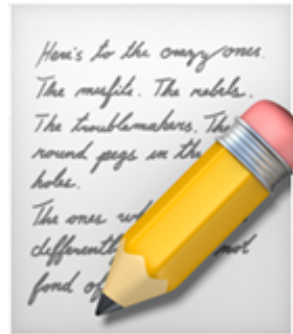
Richtig!



Welcher **eine** Gedanke zu



„Mehrere Exceptions“ war
für Sie am wichtigsten?



Schreiben Sie ihn auf!



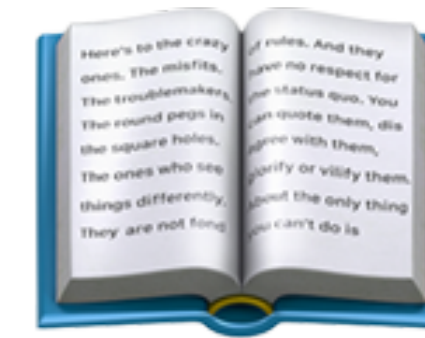
Durchatmen und



Sortieren

Polymorphe Exceptions

**Jetzt Seiten 436
und 439
wiederholen!**



Sie haben dafür 9 Minuten Zeit.



„Genau, Sie können eine Superklasse aller Exceptions deklarieren, die ausgelöst werden. IOException ist ein gängiges Beispiel, da viele I/O-spezifische Exceptions von ihr erben.“

„Und all das basiert darauf, dass Exceptions Objekte sind? Also kann ich Exceptions polymorph referenzieren, wie alle guten Objekte?“

Noel – neugierige Student:in im 1. Semester




```
1 import java.io.IOException;
2 import java.nio.file.AccessDeniedException;
3 import java.nio.file.FileSystemException;
4 import java.nio.file.Files;
5 import java.nio.file.NoSuchFileException;
6 import java.nio.file.Paths;
7 import java.text.NumberFormat;
8 import java.text.ParseException;
9 import java.util.Locale;
10
11 public class PolyExceptionsDemo {
12
13     // Declare the *two* checked exceptions that might be thrown:
14     public static Number readAndParseUSD(String filePath) throws IOException, ParseException {
15         // Might throw IOException:
16         String content = Files.readString(Paths.get(filePath)).trim();
17
18         NumberFormat currencyFormat = NumberFormat.getCurrencyInstance(Locale.US);
19
20         // Might throw ParseException:
21         return currencyFormat.parse(content);
22     }
23
24     public static void main(String[] args) {
25         String filePath = "money-uh-oh.txt";
26
27         try {
28             Number amount = readAndParseUSD(filePath);
29             System.out.println("The amount is: " + amount);
30         } catch (NoSuchFileException e) {
31             // java.lang.Exception <- java.io.IOException
32             // <- java.nio.file.FileSystemException <- java.nio.file.NoSuchFileException
33             System.err.println("Could not read from file that does not exist: " + e.getClass().getName());
34             System.err.println("Message: " + e.getMessage());
35         } catch (AccessDeniedException e) {
36             // java.lang.Exception <- java.io.IOException
37             // <- java.nio.file.FileSystemException <- java.nio.file.AccessDeniedException
38             System.err.println(
39                 "Could not read from file due to a file system operation that was denied: "
40                 + e.getClass().getName());
41             System.err.println("Message: " + e.getMessage());
42         } catch (FileSystemException e) {
43             // java.lang.Exception <- java.io.IOException
44             // <- java.nio.file.FileSystemException
45             System.err.println(
46                 "Could not read from file due to file system operation failure: "
47                 + e.getClass().getName());
48             System.err.println("Message: " + e.getMessage());
49         } catch (IOException e) {
50             // java.lang.Exception <- java.io.IOException
51             System.err.println("Could not read from file due to: " + e.getClass().getName());
52             System.err.println("Message: " + e.getMessage());
53         } catch (ParseException e) {
54             // java.lang.Exception <- java.text.ParseException
55             System.err.println("Could not parse file contents due to: " + e.getClass().getName());
56             System.err.println("Message: " + e.getMessage());
57         }
58     }
59 }
60
```

STDIN

Input for the program (Optional)

Output:

Could not read from file that does not exist: java.nio.file.NoSuchFileException
Message: money-uh-oh.txt

! ? Quiz

🎨 Polymorphe Exceptions

🤔 Ich mache einige Aussagen –
sind diese Aussagen richtig
oder falsch?

👍 Bei **richtigen** Aussagen **stehen**
Sie **kurz auf!**

👎 Bei **falschen** Aussagen **bleiben**
Sie **sitzen!**



Sie können Exceptions deklarieren, indem Sie die Superklasse der geworfenen Exceptions verwenden.

Richtig!

Die Reihenfolge der catch-Blöcke spielt grundsätzlich keine Rolle.

Die am weitesten gefassten Exceptions müssen vom letzten catch-Block abgefangen werden.

Sie können Exceptions abfangen, indem Sie eine Unterklasse der geworfenen Exception verwenden.

Der catch-Block muss die geworfene Exception oder eine ihrer Superklassen angeben.

Nur weil Sie alles mit einem großen, superpolymorphen catch-Block abfangen können, heißt das noch lange nicht, dass Sie es auch tun sollten.

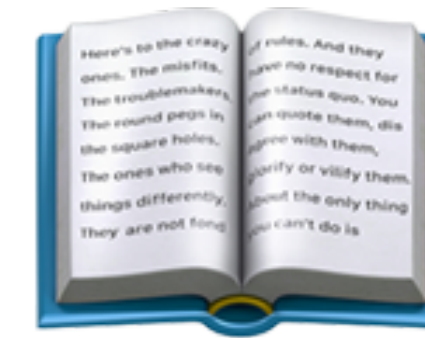
Richtig!

 Welcher **eine** Gedanke zu
„ Polymorphe Exceptions“ war
für Sie am wichtigsten?
 Schreiben Sie ihn auf!

 Durchatmen und  Sortieren

 **Ausweichen**

**Jetzt Seiten 441
und 442
wiederholen!**



Sie haben dafür 5 Minuten Zeit.



„Im Prinzip ja, aber bedenken Sie, dass an irgendeiner Stelle die Exception behandelt werden sollte, da sonst die JVM abrupt beendet werden könnte.“

„Wenn meine Methode mit throws eine Exception deklariert, dann brauche ich dafür keinen try/catch-Block und weiche so elegant aus.“

Merle – aufmerksame Student:in im 1. Semester



! ? Quiz

↩ Ausweichen

- 🤔 Ich mache einige Aussagen – sind diese Aussagen richtig oder falsch?
- 👍 Bei **richtigen** Aussagen **stehen** Sie **kurz auf**!
- 👎 Bei **falschen** Aussagen **bleiben** Sie **sitzen**!



Wenn Sie eine Methode schreiben, die eine vom Compiler überprüfte (checked) Exception auslösen könnte, müssen Sie den riskanten Code zwingend mit einem try/catch-Block umgeben.

Sie können die Exception deklarieren und somit „ausweichen“.

Die Behandlung einer Exception mittels try/catch-Block wird manchmal auch als „Ausweichen“ bezeichnet.

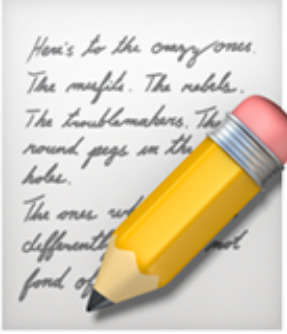
„Ausweichen“ ist gleichbedeutend mit „Deklarieren“.

Wollen Sie eine Exception nicht selbst behandeln, können Sie ihr ausweichen, indem Sie sie mit `throws` deklarieren.

Richtig!

Die `main()`-Methode Ihres Programms muss zwingend alle unbehandelten Exceptions, die ihr „zugeworfen“ wurden, behandeln.

Sie kann ausweichen. Aber wenn sie das tut, wird die JVM möglicherweise beendet.

 Welcher **eine** Gedanke zu
„ Ausweichen“ war für Sie
am wichtigsten?  Schreiben
Sie ihn auf!



Durchatmen und  Sortieren



Exception-Regeln



Behandeln 🧑‍🚒 oder Ausweichen 💣, das ist hier die Frage

🧑 Exception-Regeln

```
try {  
    URL url = new URL(input);  
} catch (MalformedURLException e) {  
    // handle exception  
}
```

🧑 Den riskanten Aufruf
in einen try/catch-
Block verpacken!

💣 Die gleichen Exceptions deklarieren, wie
- oder - sie die aufgerufene riskante Methode auslöst!

```
URL constructURL(String url) throws MalformedURLException {  
    return new URL(url);  
}
```


Kombinationen von try/catch/finally

Exception-Regeln

① Kein catch oder finally ohne ein try.

```
void go() {  
    Foo f = new Foo();  
    f.fooof();  
    catch(FooException ex) { }  
}
```

UNGÜLTIG!
Wo ist der
try-Block?

② Zwischen try und catch darf sich kein anderer Code befinden.

```
try {  
    x.doStuff();  
}  
int y = 43;  
} catch(Exception ex) { }
```

UNGÜLTIG! Zwischen
dem try- und dem
catch-Block darf kein
anderer Code stehen.

③ Auf ein try MUSS entweder ein catch oder ein finally folgen.

```
try {  
    x.doStuff();  
} finally {  
    // aufräumen  
}
```

GÜLTIG, weil es einen finally-
Block gibt, auch wenn nichts
abgefangen wird. Ein allein
stehender try-Block funk-
tioniert dagegen nicht.

④ Ein try, das nur einen finally-Block besitzt, muss die Exception trotzdem deklarieren.

```
void go() throws FooException {  
    try {  
        x.doStuff();  
    } finally { }  
}
```

Ein try ohne catch
befolgt nicht das
>>Behandeln oder
Deklarieren<<-Gesetz.

! ? Quiz

👤 Exception-Regeln

- 🤔 Ich mache einige Aussagen – sind diese Aussagen richtig oder falsch?
- 👍 Bei **richtigen** Aussagen **stehen** Sie **kurz auf**!
- 👎 Bei **falschen** Aussagen **bleiben** Sie **sitzen**!



Wenn Sie einen riskanten Methodenaufruf ausführen – es könnte eine geprüfte Exception geworfen werden – dann müssen Sie entweder behandeln (try/catch-Block) oder ausweichen (mit throws deklarieren).

Richtig!

Laufzeit-Exceptions müssen behandelt oder deklariert werden.

Laufzeit-Exceptions sind ungeprüfte (unchecked) Exceptions.

Ein `finally`-Block kann ohne einen `try`-Block existieren.

Einem `finally`-Block geht immer ein `try`-Block voraus.

Ein `catch`-Block kann nur mit einem `try`-Block existieren.

Richtig!

Ein try-Block kann auch ohne einen catch- oder finally-Block existieren.

Einem try folgt entweder ein catch oder ein finally – oder beides.

Eine Methode mit einem try-Block und einem finally-Block braucht eine eventuell geworfene geprüfte (checked) Exception nur optional zu deklarieren.

Ohne catch-Block müssen Sie deklarieren („ausweichen“).

 Welcher **eine** Gedanke zu
„ Exception-Regeln“ war für
Sie am wichtigsten?
 Schreiben Sie ihn auf!

 Durchatmen und  Sortieren

 **Zielgerade**

↔ Teach Back

📖 Exception-Handling

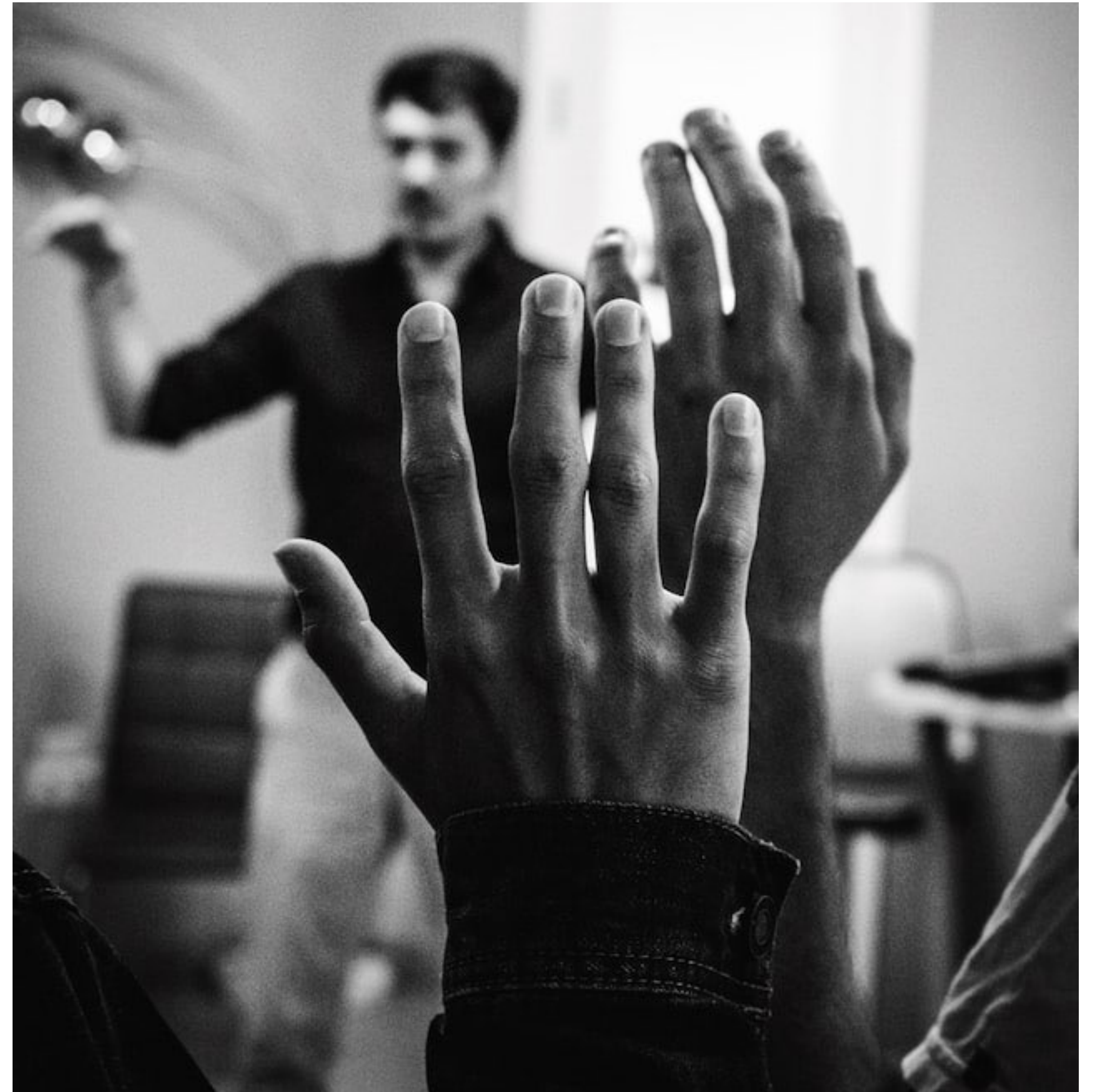
- 🎤 Beantworten Sie sich zusammen mit einer Nachbar:in gegenseitig kurz und knapp die folgenden Fragen:
- 💡 Was habe ich heute gelernt und erfahren?
- 💡 Was davon ist für mich besonders wichtig?



💡 Ihre Fragen

👉 Unsere Antworten

- Welche Fragen haben Sie zu  **Exception-Handling**?
- Was irritiert Sie vielleicht sogar?
- Was interessiert Sie außerdem?





Lern-Logbuch



Exception-Handling



Machen Sie sich Notizen:



Was sind für mich die wichtigsten Erkenntnisse?



Was weiß ich jetzt, was ich vorher nicht wusste?



Wie kann ich dieses Wissen anwenden?



Welche Fragen habe ich noch?



Was werde ich tun, um sie zu beantworten?



Ihr Lernerfolg – Unser Applaus

